Google Developer Group
Indian Institute of Technology Indore

Inductions 2026

Deadline: 23rd January 2025 (Friday) 11:59 PM IST

Instructions:

- ***This is an individual task; collaboration leads to immediate disqualification.***
- ***Create a GitHub account here.***
- ***Good Coding Practices: Naming Conventions, Meaningful Variable and File Names, Appropriate Folder Structure, Code Readability, and Comments are encouraged.***
- ***Each task is self-contained; interpretations may vary, and there is no one "answer" to a single problem.***
- ***Make sure to add a README file in your GitHub repository consisting of:***
  - ***Brief task description***
  - ***Related screenshots***
  - ***Instructions to run the project***
  - ***If hosted, its URL***
  - ***Any external Links(If the PS demands)***
- ***Submit the code even if the task is incomplete.***
- ***Create a public repository for your task and submit the link in the form.***
- ***Task prioritization underscores the emphasis on challenging assignments, with evaluation leaning towards recognizing and rewarding diligent efforts in handling more intricate tasks over simpler ones.***

For any queries, contact:

S Varshith(+9191216 60920)
Web Development

Satyam (+91 88301 20040)
AI/ML

Ninad(+91 7888040312)
Blockchain

Anuskha (+91 81470 30408)
Web Development

Abhishek (+91 79033 26287)
App Development

Sarath (+91 88299 93469)
Game Development

Web Development

GDG Web Development Division - Induction Tasks

Problem Statement 1: (Frontend)

Pokedex - Web

Problem Statement:

Create a responsive frontend only poke dex (If you don't know what it is, read here). You need to fetch the data from an external API and manage the response and render the data received on a clean UI. The fetching of the pokemon details can be done from the Official Poke API (Link) where you can fetch the required data using multiple API endpoints available.

Key Features:

- **General pokemon display page:** The API is a really vast resource for pokemon related details, and you are free to implement whatever you like but here are some features that must be there.
- **Pokemon Search Feature:** The user must be able to search for a pokemon name and be able to see all the details related to their search.
- **Pokemon Display:** The UI must also show pokemon like a gallery of pokemon taken from the API. The UI must properly display the details received from the API and must handle errors and empty responses as well.
- **Other details page:** The API does provide endpoints for other things related to the pokemon universe, so it's up to your creativity and imagination to decide what to display in your application.

Useful Resources:

Here is the link for the official PokeAPI documentation.

Here is the link for the official React JS documentation

Here is a video resource for React Js.

A Few Pointers:

- **Brownie points for a well-designed UI**
  Since this is a *frontend-centric task*, emphasis will be placed on visual clarity, layout structure, responsiveness, and overall user experience.
- **Robust error handling**
  Proper handling of API failures, network issues, and unexpected responses is expected.
- **Attention to edge cases**
  Handling empty states, loading transitions, and partial data gracefully will be considered a strong signal.

Problem Statement 2: (Frontend)

Expense Splitter

Problem Statement:

When groups of friends spend money together, calculating who owes whom often becomes confusing and error-prone. People usually end up doing manual calculations, which leads to mistakes and misunderstandings.Your task is to build a frontend-only web application that helps users split expenses fairly and transparently. The application should allow users to record expenses and automatically compute final balances so that everyone clearly understands how much they owe or are owed.

Key Features

- Create a group with multiple members.
- Add expenses with:
    - Expense name
    - Amount
    - Person who paid
    - People involved in the expense
- Automatically calculate:
    - Total amount spent by each person
    - Net balance (how much each person owes or is owed)
- Display a clear summary of settlements.
- Persist all data using localStorage so that data is not lost on refresh.

Extra Information

- No backend or APIs.
- No authentication required.
- Correct calculations are more important than UI design.

Resources:

- If hosting on github or vercel here is a short guide.
- Pointers to using local Storage on any browser (Here)

Problem Statement 3: (Backend)

Task Manager Backend API

Problem Statement:

Design and implement a Rest API for managing tasks. The API must be able to accept tasks and edit them and also change their status, the data from the user must persist and also the user must authenticate before adding any tasks so whenever they try to request something to the API an authentication check must be made.

Key Features:

- **Task management:** The user must be able to add, edit and change the status of tasks which they have added previously.
- **User Authentication:** The user must be able to create an account in the application and save their tasks under their account.
- **Saving Data:** The data must persist between API calls so that the user can leave and when they use the application later they must be able to see their data intact, i.e usage of a database to persist data is a must.
- **Bonus:** Implement task reminders using email notifications (Nodemailer, SendGrid).

Resources:

Backend: use any one of your choice, Node JS or Python based fast API and/ or django, flask.

Databases: use any one of your choice, MongoDB or any SQL based database.

API Protocol: Rest Protocol based API.

Other resources related to backend development have already been shared. (Link)

Problem Statement 4: (Backend)

URL Shortener Service

Problem Statement:

Long URLs are difficult to share and remember. URL shorteners solve this problem by converting long links into shorter, easy-to-share URLs that redirect to the original destination.

Your task is to build a backend service that provides basic URL shortening functionality. The service should accept long URLs, generate short unique identifiers for them, and redirect users when the short URLs are accessed.

Key Features:

- **Create endpoint:** There must be an endpoint for the creation of the short URL given a long URL
- **Redirect endpoint:** API endpoint to redirect a short URL to the original URL.
- **Duplication prevention:**Prevent creating multiple short URLs for the same long URL.
- **Analytics:** Track how many times each short URL is accessed.
- **Data persistence:** Store data persistently using a database or file-based storage.
- **Bonus:** There is no need for authentication and a frontend, but a light weight frontend and authentication (bonus) can fetch you some brownie points.

Resources:

Backend: Use any NodeJS based server or any python based backend ecosystem.

Database: Use any MongoDB or SQL based database.

API Protocol: Rest Protocol based API.

Other resources related to backend development have already been shared. (Link)

Problem Statement 5: (Fullstack)

Issue/ Bug tracker for developers

Problem Statement:

Design and create an application where users (developers) can keep track of bugs and issues present in their codebases. Note that saving the code bases in the application is not necessary, but in the application you must be able to create, assign and edit the status of multiple issues and assign them to others etc. Conceptually this can be considered similar to Jira which is one such system, what we want here is a simpler and light weight version of Jira or any such system.

Key Features:

- **User Authentication:** The users must be able to create accounts via credential or Google Oauth.
- **Issues:** The users must be able to create issues, assign them to other people and also comment/ continue a thread on some open conversation under an issue.
- **UI:** The users must be able to see issues assigned to them and all details related to them in a clean kanban-board style UI (Article), for cleaner looks, but it is not mandatory and other UI styles are also accepted.
- **Priority Setting:** The users must be able to assign priority to the issues they raise and also set a priority to the list of issues they have been assigned with.
- **Bonus:** Implement an email system to send an email when the user has been assigned an issue, fetching details from github APIs can add a ton of reliability and showcase skills, so try to do so (not mandatory).

Resources:

Resources related to full stack web development and usage of databases have already been shared (here).

Database: Use any one of mongoDB or any sql based database.

Backend,Frontend: Use any NodeJS or python based system, react JS or any other

Problem Statement 6: (Fullstack)

Task Board (Mini Trello)

Problem Statement:

Task tracking is a common requirement in real-world software systems, used by individuals and teams to organize and monitor work. Most task management tools are built around tasks moving through different stages of completion.Your task is to build a simple full-stack task board application where users can create tasks and move them through different stages such as pending, ongoing, and completed.

Key Features

- Three task columns:
  - To Do
  - In Progress
  - Done
- Create, edit, and delete tasks.
- Move tasks between columns.
- Backend APIs to store and update tasks.
- Frontend fetches tasks from the backend and reflects updates correctly.
- Task data persists after page refresh.
- Optionals, can add a multiple user shared tracking system
- Authentication is optional as well, but would fetch you some credit if done properly.
- Drag and drop UI style is preferred but is not necessary.

Resources:

Resources related to full stack web development and usage of databases have already been shared (here).

Database: Use any one of mongoDB or any sql based database.

Backend,Frontend: Use any NodeJS or python based system, react JS or any other.

Machine Learning

GDG: AI/ML Division - Problem Statements

General Instructions & Guidelines

1. **Concepts over Completion:** AI tools are allowed for learning and coding to some extent, **but this will weaken your induction chances if you cannot explain the code or logic correctly.** We evaluate your grasp of the engine, not just the driving.
2. **Product Mindset:** Don't just write code; build a usable, robust product that solves user pain points.
3. **Submission Checklist:**
   - **Compulsory:** PS1 (Task 1 & 2).
   - **Optional:** PS1 (Task 3) and PS2 (Task 1).
   - *Note:* Doing optional tasks will significantly boost your induction chances.
4. **Prerequisite:** You must complete the compulsory tasks before attempting the optional ones. Optional tasks will only be evaluated if the base requirements are met.
5. **No "Vibe Coding":** For PS2 (Diffusion), you must be able to explain the process. Blindly copying code without theoretical backing will result in disqualification. **No need to train the model to convergence, but the logic must be correct.**
6. **Zero Plagiarism:** Originality is key. Fresh ideas and unique implementations are your biggest assets.
7. **Direct Induction:** Exceptional, novel work (e.g., unique architecture or optimizations) beyond the basic requirements can fast-track you to the team.
8. **Explore:** The provided resources are just a start. Research and implement ideas beyond the docs.

> **"Submit whatever work you have done. Something is always better than nothing!"**

Problem Statement 1: Capital Pulse - AI Driven Stock Market Intelligence

Motivation

Everyone wants to make free money. They invest in the stock market using their father's hard-earned income and often fail to convert it into profits. But for the next few weeks, you will try to rely not just on intuition, but on logic and a taste of ML.

> *"Data without insight is noise. The future of fintech belongs to systems that can not only foresee market shifts but explain the 'why' behind the numbers."*

Financial markets are complex systems driven by historical patterns and real-time events. Traditional models often provide numbers without context, leaving users to guess the reasoning. The goal is to build a system that bridges this gap.

Participants are required to build a **Dual-Core Financial Intelligence System** that acts as both a **Quantitative Analyst** (forecasting trends) and a **Financial Advisor** (explaining the "why"). The system should take raw financial data and turn it into actionable, explainable insights using open-source technologies.

Task Breakdown

Participants must complete the following tasks to demonstrate a complete pipeline.

Task 1: The Predictive Core (Static Time Series)

Goal: Develop a forecasting engine that learns from historical market data to predict future trends for a specific period. The goal is not just to draw a line, but to model the uncertainty and volatility inherent in financial assets.

- **Input:** A CSV file serving as a database containing the past few years of daily stock data for a specific ticker (e.g., OHLCV data). Multiple tickers can be supported.
- **Process:** Train or utilize a pre-trained model to analyze seasonal trends and price momentum.
- **Output:** A dashboard displaying:

- Historical trend graph.
- Projected line for the next 7 days (including plotting predictions for past data).
- Shaded "confidence interval" area indicating potential high/low variance.
- Accuracy metrics (calculated on past data).

Ideas for Success:

- Use frameworks like **PyTorch**, **TensorFlow**, or **Statsmodels**.
- Implement rolling forecast validation.
- Fine-tune pre-trained models on financial datasets.
- Experiment with mathematical approaches like AR, MA, or VAR models.

Task 2: The Analytical Chatbot (RAG & Explanation)

Goal: Create a natural language interface that allows users to query the dataset and understand the context behind the numbers. The chatbot must identify specific tickers and retrieve relevant information to explain market movements.

- **Contextual Query Example:** User asks, *"Why did Apple stock drop?"* → System identifies 'AAPL', retrieves news/reports, and explains the cause.
- **Trend Analysis Example:** User asks, *"When did Microsoft go up?"* → System analyzes 'MSFT' data, finds uptrends, and correlates them with events.

Ideas for Success:

- **News Tool:** Scrape/search financial news to populate a vector database for cited answers and apply sentiment analysis for better predictions.
- **Agent Architecture:** Distribute tasks (e.g., "Plotting Agent" vs. "Research Agent").
- **RAG:** Use Query Rephrasing, Reranking, and Splitting.
- **LLMs:** Use open-source models (Hugging Face) or APIs (Groq, Gemini). Orchestrate with LangChain.
- **Intent:** Semantically understand user intent rather than using simple keywords.

Task 3: Real-Time Intelligence (Live Data Integration) *(Optional)*

Goal: Upgrade the static system to handle live data ingestion. The system should maintain a connection to a market data source and update its analysis in real-time without refreshing the page.

- **Data Integrity:** Backfill missing historical data (e.g., via scraping) before the live stream starts to ensure continuity.
- **Live Dashboard:** Chart updates every minute as new price data arrives via

WebSocket/API.

- **Dynamic Alerts:** Trigger notifications if live prices cross predicted thresholds.

Resources:-

1. https://youtube.com/playlist?list=PLvcbYUQ5t0UHOLnBzl46_Q6QKtFgfMGc3&si=RNcLrTiVAlrw1d2H
2. Build a Stock Trend Prediction Web App in Python | GeeksforGeeks
3. Fetching Live and Historical TradingView Data in Python: No External Modules Re…

Problem Statement 2: The Pixel Alchemist - Decoding Diffusion *(Optional)*

Motivation

You might have seen how AI images are made, but have you ever thought about the core process that makes them do that? Here we are with a challenge to look under the hood.

> *"Understanding the canvas before painting on it. True mastery of Generative AI comes not from calling APIs, but from understanding the mathematics of noise."*

This problem statement tests your theoretical understanding and implementation skills regarding **Probabilistic Diffusion Models**. The goal is to strip away abstractions and build the core components from scratch.

Constraint: Solutions must demonstrate a clear understanding of the underlying math. "Vibe coding" (blindly copying code or relying entirely on high-level wrappers without understanding) is discouraged.

Task Breakdown

Task 1: Diffusion from Scratch (Theory Based)

Goal: Implement a basic diffusion model (DDPM or similar) from first principles using a framework like PyTorch (Refer resources). **You cannot use the diffusers library for the model architecture or training loop.**

Requirements:

1. **Forward Process:** Implement a noise scheduler (linear or cosine) that gradually adds Gaussian noise to an image.
2. **Reverse Process:** Design and implement a U-Net (or similar autoencoder) to predict the noise added to the image.
3. **Sampling:** Implement the sampling algorithm to generate images from pure

noise.

4. **Demonstration:** Provide a script demonstrating the forward noise addition and a functional sampling loop (even if the output is noisy due to limited training).

Resources:-

1. Coding Stable Diffusion from scratch in PyTorch
2. But how do AI images and videos actually work? | Guest video by Welch Labs

Deliverables

Submission Structure

Participants must submit a single GitHub repository structured as follows:

```
├── README.md      (Comprehensive walkthrough for the entire project)
├── PS1/
│   ├── Task1/     (Predictive Core code)
│   ├── Task2/     (Chatbot code)
│   └── Task3/     (Live Integration code)
└── PS2/          (Optional)
    └── Task1/     (Diffusion Scratch code)
```

Detailed Requirements

- **Source Code:** Clean, modular code organized strictly into the subfolders above.
- **The Application (PS1):** A functional dashboard toggling between "Static Analysis" and "Live Market Mode."
- **The Logic (PS2):** Scripts must be well-commented to explain the logic, especially for the scratch implementation. Don't make it clumsy!
- **README.md:** A detailed walkthrough explaining model choices, pipeline architecture, and data management including how to run your codebase.

ALL THE BEST GUYS!

App Development

1. Beginner Level – Calculator App

PS (Problem Statement)

People often need a simple tool to perform quick arithmetic operations, but many calculator apps include unnecessary functions that confuse beginners. This app focuses on providing a minimal, intuitive calculator for accurate and efficient basic calculations.

Key Features

- Basic arithmetic operations: Addition, Subtraction, Multiplication, Division
- Clear (C) and All Clear (AC) buttons
- Simple and clean interface
- Continuous calculations
- Error handling (e.g., divide-by-zero)
- Optional: Dark/Light theme
- Optional: Scientific calculator mode
- Optional: Any advanced feature which is Different from other calculators

Deliverables

- Fully functional calculator app
- Clean and user-friendly UI
- Working continuous operations and error handling
- Optional theme toggle or scientific mode
- Source code and a brief README

Resources

- Java/Kotlin (Android) : Kotlin and Android | Android Developers

Youtube video for beginners: Full 2025 Kotlin Crash Course For Beginners

How to Build a Calculator with Jetpack Compose - Android Studio Tutorial

- React Native : Introduction · React Native or KMP (Multiplatform) : Resources overview | Kotlin Multiplatform Documentation
- Android Studio or Xcode or any other suitable platform

2. Intermediate Level – Shopping List App

PS (Problem Statement)
People often forget important items while shopping or buy unnecessary things due to lack of planning. A shopping list app helps users stay organized, track what they need, and avoid overspending.

Key Features
- Add, edit, and delete shopping items
- Mark items as purchased
- Categorize items (Grocery, Electronics, Personal Care, etc.)
- Quantity and notes for each item
- Sort items by category or priority
- Optional: Reminders/notifications
- Optional: Share list with others or Any advanced feature which is Different from other shopping app

Deliverables
- Complete shopping list app with CRUD functionality
- Categorization and sorting features
- UI to manage items and purchased lists
- Optional reminders or sharing feature
- Source code and README

Resources
- Local storage (Room Database or SharedPreferences) :
The FULL Beginner Guide for Room in Android | Local Database Tutorial for Android
- Material Design components : Components — Material Design 3
- Notification API (optional)
- Firebase or sharing methods (optional)

3. Advanced Level – Recipe Finder App

PS (Problem Statement)
People often struggle to decide what to cook based on available ingredients. Manually searching for recipes takes time. This app suggests recipes instantly based on available ingredients, reducing food waste and making cooking easier.

Key Features
- Input ingredients
- Recipe suggestions based on ingredients
- Detailed recipe steps and ingredients
- Filters: veg/non-veg, cuisine, difficulty
- Save favorite recipes
- Optional: Random or daily recipe
- Optional: Advanced or AI-based recommendations

Deliverables
- Fully functional recipe finder app
- Ingredient input and recipe matching system
- Recipe detail screens
- Filtering options
- Favorite recipes section
- Optional random/daily recipe or Any advanced feature which is Different from other Recipe app
- Source code and documentation

Resources
- Open recipe APIs (Spoonacular, Edamam, or any free one): Nokia acquires Rapid technology and team! 🚀
- Retrofit or any HTTP client
- MVVM or similar architecture:

ViewModels & Configuration Changes - Android Basics 2023

- Firebase or local database for favorites:

Firebase Google Sign-In With Jetpack Compose & Clean Architecture - Android S...

Blockchain

Level 1: Decentralized Vault (Basic)

Objective: Implement a basic DeFi "Vault" contract that handles native currency (ETH) deposits and withdrawals.

Technical Requirements:

- **Deposit Logic:** Implement appropriate functions to allow users to send ETH to the contract. The contract must track individual balances using a mapping.
- **Withdrawal Logic:** Implement appropriate functions to allow users to withdraw their specific deposited amount.
- **Security:** Use appropriate logic to prevent a user from withdrawing more than their current balance using validation statements.
- **State Management:** Ensure the user's balance is updated in the mapping before funds are transferred to mitigate potential re-entrancy risks.
- **Resources**:
  - **Solidity by Example:** Common Patterns and Logic
  - **Solidity Tutorial (Core Concepts)**

    Solidity Programming Tutorial Hindi | Solidity - Ethereum | Code Eater - Blockchain

    ERC-20 (Fungible Tokens): OpenZeppelin ERC-20 Documentation

Level 2: Decentralized Governance (Basic)

Objective: Develop a smart contract for transparent, on-chain voting.

Technical Requirements:

- **Candidate Initialization:** Implement appropriate logic to define a list of candidate names upon contract deployment.
- **Voting Mechanism:** Implement appropriate functions to allow users to submit a vote that increments a candidate's tally.
- **Access Control:** Utilize a mapping to ensure a single address is restricted to exactly one vote.
- **Transparency:** Include appropriate functions to return the current standings or results of all candidates.
- **Resources**:
    - **Faucet**:https://cloud.google.com/application/web3/faucet/ethereum/sepolia

Level 3: Advanced NFT-Gated Application (Advanced)

Objective: Build a full-stack decentralized application (dApp) that utilizes the NFT deployed during the introductory lecture for access control.

Technical Requirements:

- **Smart Contract Interaction:** Integrate a frontend using **Ethers.js** or **Web3.js** to communicate with the existing ERC-721 contract.
- **Wallet Integration:** Implement a secure "Connect Wallet" flow for user authentication.
- **Gated Logic:** The application must utilize appropriate functions to check the user's NFT balance. Access to restricted areas should only be granted if the balance meets the required threshold.
- **Professional Testing:** * **Extra Credit:** Provide a comprehensive test suite using **Foundry**.
    - Tests must verify that unauthorized users are successfully blocked and that the contract state remains consistent after failed transactions.
- **Resources**:
    - **ERC-721 (Non-Fungible Tokens):** OpenZeppelin ERC-721 Documentation
    - **Ethers.js (v6):** Official Documentation
    - **Ethers.js Tutorial (Frontend Integration)**
    - Ethers.js Course | Ethereum Library | Web3.js vs Ethers.js | Code Eater - B…
    - **Foundry:** Foundry Book - Installation and Testing
    - **Foundry – Introduction & Tooling** *(Advanced)*
    - Intro to Foundry | The FASTEST Smart Contract Framework

---

Deliverables

- **Public GitHub Repo:** All code included with a proper README describing the approach used.Include the links of your deployed contracts on etherscan or other explorer

Grading

- **Code:** Correctness, security, and optimization.
- **Approach & Understanding:** Rationale behind logic and clarity of documentation.

ALL THE BEST!

Game Development

PS 1 : Endless Arcade Game (Beginner)

Problem Statement:

Design and develop an **endless arcade-style game** of your choice.

You may take inspiration from classic arcade or endless games (such as Flappy Bird, Temple Run, Subway Surfers, etc.), but **original ideas or unique twists on the genre are encouraged** and will receive bonus consideration.

Resources:

- Endless Runner tutorial: Link
- Game Development Resources: Link

Requirements:

- The game must be endless (no fixed final level).
- The game must include a scoring system (time, distance, points, etc.) and a high score.
- The game should gradually increase in difficulty over time.
- The player must have a clear way to lose and restart the game.

Final Deliverables:

The final submission must include a GitHub repository link containing the following:

- The source files of the game.
- A README file containing:
    - Setup and gameplay instructions
    - A list of any games used as inspiration (if applicable).
    - A drive link to the final build of the game (executable file).

Including instructions within the game itself is encouraged.

PS 2: Theme - Based Game Jam (Intermediate)

Problem Statement:

Make a game that takes and implements 2 out of the following **6 themes**:-

- **Shadows**
- **Divide**
- **Simultaneous Control**
- **Shapes**
- **Navigation**
- **Now You See Me, Now You Don't**

NOTE: These are some open ended themes which can be incorporated in the games in a number of ways. Use your own creativity to incorporate the themes.

The selected themes must be reflected in the game's **core design**, influencing gameplay behavior, progression, or overall player experience. Themes may be interpreted mechanically, systemically, narratively, or through a combination of these approaches.

Purely cosmetic, or narrative-only interpretations that do not **meaningfully impact** gameplay are discouraged.

You should be able to clearly explain how your design choices express the chosen themes.
Resources: This link contains general game development resources. This PS has no specific tutorials, **creativity is your only limit!**

Evaluation Criteria:

Submissions will be evaluated based on the following criteria:

- Depth and clarity of theme integration — **30%**
- Coherence and consistency of gameplay — **20%**
- Creativity and originality in interpreting the themes — **30%**
- Gameplay design and execution — **20%**

Final Deliverables:

The final submission must include a GitHub repository link containing the following:

- The source files of the game.
- A README file containing:
    - Setup and gameplay instructions
    - The two selected themes
    - A brief explanation of how each theme is incorporated into the game
    - A drive link to the final build of the game (executable file).

Including instructions within the game itself is encouraged.

ALL THE BEST!